Mocking Distributed Databases Located in Different Geographical Regions Using Docker Containers

Suraj Joshi *
Student ID: 2024280205
Department of Computer Science and Technology, Tsinghua University

Klisiewicz Dominik Stanislaw
Student ID: 2024400515
Exchange Student, Tsinghua University

December 31, 2024

Abstract

This report covers the design and implementation of a distributed database system simulated using MongoDB on Docker containers, aiming to replicate the behavior of distributed databases across geographical regions. The system employs MongoDB's sharded cluster architecture, featuring a three-node configuration server replica set, two shard replica sets, and a MongoDB router (mongos), with Redis integrated for caching and GridFS for efficient storage of large unstructured files like images and videos. A simple user interface facilitates interaction with the database, which manages structured data across five relational tables and unstructured data (text, images, video). Data fragmentation and distribution strategies are based on entity attributes, such as region for user data, category for article data, and temporal granularity for popularity ranking data, ensuring efficient data placement. The project highlights the strengths of NoSQL solutions like MongoDB in handling heterogeneous data, showcasing a scalable and flexible approach to distributed data management and providing insights for future improvements and real-world applications.

Keywords: Distributed Database Systems, MongoDB, Docker, Sharded Cluster, NoSQL, Redis, GridFS, Data Fragmentation

*Project Leader, Corresponding author: surz24@tsinghua@mails.edu.cn

Problem Background and Motivation

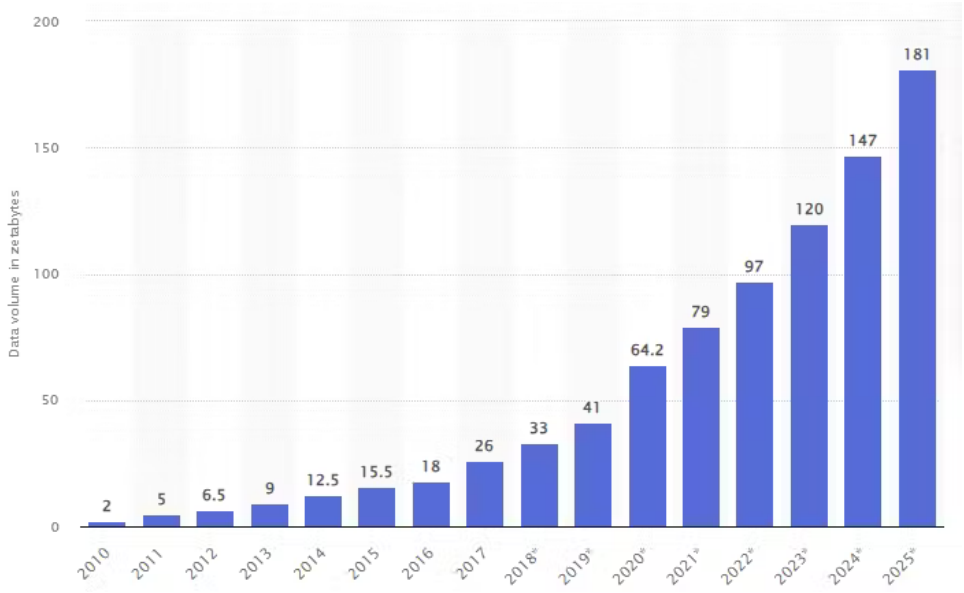


Figure 1: Data growth trends (Source: Statista)

From the Figure 1, it is clear that the amount of data created, captured, copied, and consumed worldwide has been growing at an exponential rate. Traditional centralized databases have served as the backbone for data storage and management for decades. However, their inherent limitations have become increasingly evident with the rapid growth in data volume, variety, and velocity. These limitations include:

Challenges of Centralized Databases

- **Single Point of Failure**: Centralized databases are vulnerable to failures, which can result in complete system downtime and loss of accessibility.
- **Scalability Constraints**: Scaling centralized systems often requires expensive hardware upgrades and may not adequately address increasing demands.
- **Performance Bottlenecks**: A single server handling all queries and transactions can become a bottleneck, leading to delays and inefficiencies.
- **Limited Availability**: Centralized systems may struggle to provide continuous availability, particularly during maintenance or unexpected failures.
- **Geographical Limitations**: Accessing a centralized database from remote locations can lead to increased latency and degraded performance for global users.

How Distributed Databases Address These Challenges

Distributed databases offer a robust solution to the problems associated with centralized systems by distributing data across multiple nodes in a network. This approach provides:

- Enhanced fault tolerance through data replication and redundancy.
- Improved scalability by allowing horizontal expansion with additional nodes.
- Increased performance through parallel query processing and load balancing.

- High availability ensured by distributed failover mechanisms and continuous monitoring.
- Reduced latency and better performance for geographically dispersed users.

Existing Solutions

There are many options available for distributed database solutions. The choice of a distributed database solution depends on specific application needs such as scalability requirements, consistency models, and operational complexity. Each of the databases mentioned offers unique features that cater to different use cases ranging from real-time analytics to large-scale transactional systems.

Key Distributed Database Solutions

- **Apache Ignite**
  - **Type**: Open-source, in-memory distributed database.
  - **Features**: High-speed data processing, scalable SQL support, and integration with various external databases. It offers both in-memory and disk storage options for flexibility.
- **Apache Cassandra**
  - **Type**: NoSQL distributed database.
  - **Features**: Highly available and fault-tolerant, designed for handling large datasets with low latency. It is used by major companies like Netflix and Uber due to its resilience and scalability.
- **Couchbase Server**
  - **Type**: NoSQL document-oriented database.
  - **Features**: Enhanced performance metrics and security features in its latest versions. It supports flexible data models and is suitable for applications requiring high throughput.
- **CockroachDB**
  - **Type**: NewSQL distributed database.
  - **Features**: Designed for cloud-native applications with strong consistency guarantees, automatic sharding, and multi-datacenter support. It uses the PostgreSQL wire protocol for compatibility.
- **FoundationDB**
  - **Type**: Multi-model NoSQL database.
  - **Features**: Fault-tolerant with a focus on high performance across various workloads. Its architecture allows for different data types to be stored within a single database.
- **YugabyteDB**
  - **Type**: Open-source relational database.
  - **Features**: Built for cloud-native environments, it offers horizontal scalability and low-latency querying capabilities across multiple availability zones.

- **HBase**
  - **Type**: NoSQL database modeled after Google's Bigtable.
  - **Features**: Designed to handle large datasets in a distributed manner, providing strong consistency and high availability.
- **Trino (formerly PrestoSQL)**
  - **Type**: Distributed SQL query engine.
  - **Features**: Allows querying across multiple data sources, making it suitable for big data analytics applications.
- **CrateDB**
  - **Type**: Distributed SQL database optimized for operational analytics.
  - **Features**: Hybrid storage model supporting both structured and unstructured data, ideal for IoT applications.
- **Rqlite**
  - **Type**: Lightweight distributed relational database.
  - **Features**: Built on SQLite with full replication capabilities, suitable for critical relational data storage.

Problem Definition

We are tasked with simulating a distributed database system designed to handle structured and unstructured data effectively. Data to be managed and processed include structured data (five relational tables) and unstructured data (text, images, and video). Their inter-relations are illustrated in Figure 2 . The goal is to fragment, allocate, and process data across different geographical regions while ensuring high performance, scalability, and fault tolerance.

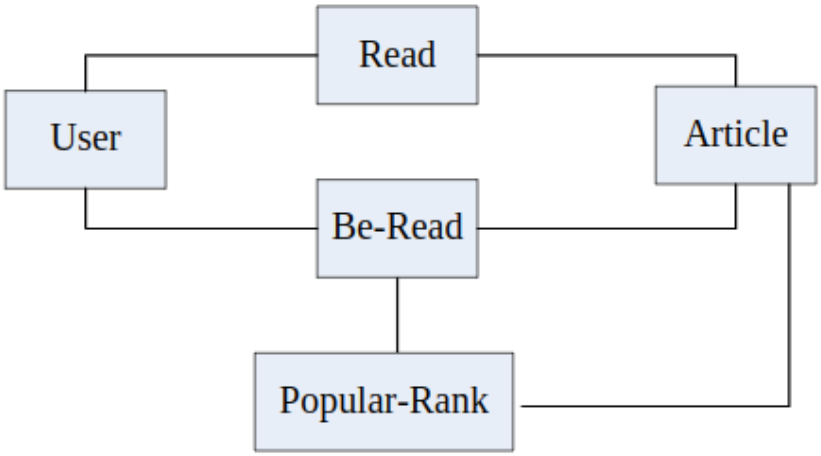


Figure 2:ER diagram of the tables

Data Description

The system must manage two types of data:

- **Structured Data**: Five relational tables (*User*, *Article*, *Read*, *Be-Read*, and *Popular-Rank*) containing structured records.
- **Unstructured Data**: Includes associated text, images, and video files related to articles, to be stored in a file system.

Table Descriptions

- **User Table**: Records user information such as ID, name, region, and activity status.
- **Article Table**: Contains article metadata, including ID, title, category, and author details.
- **Read Table**: Logs user interactions with articles, including timestamps, actions, and user IDs.

Generating Derived Tables

Based on the primary tables, the following derived tables must be populated:

- **Be-Read**: Summarizes user interactions for each article, including read counts, user lists for comments, likes, and shares.
- **Popular-Rank**: Ranks articles based on popularity within daily, weekly, and monthly timeframes.

Fragmentation and Allocation Requirements

Structured data is fragmented horizontally and allocated to two database management systems (DBMS1 and DBMS2) based on attributes such as region, category, and temporal granularity. Unstructured data is stored in a distributed file system (Hadoop HDFS). Table 1 summarizes the fragmentation and allocation scheme.

Table	**DBMS1**	**DBMS2**
User (region)	Beijing	Hong Kong
Article (category)	Science	Science and Technology
Read (region)	Beijing	Hong Kong
Be-Read (category)	Science	Science and Technology
Popular-Rank (temporalGranularity)	Daily	Weekly and Monthly

Table 1: Fragmentation and Allocation Scheme

Database Operations

The system must support the following operations:

- Bulk load of *User*, *Article*, and *Read* tables into the data center.
- Query operations across *User*, *Article*, and *Read* tables with and without filtering conditions.

- Computation and insertion of derived records into the *Be-Read* table.
- Querying the *Popular-Rank* table to retrieve top-5 articles within specified timeframes, including their associated unstructured data.

System Requirements

The distributed database system must:

- Handle data fragmentation and replication during data insertion.
- Support efficient query execution with minimal latency.
- Provide monitoring features, such as server workload, data distribution, and operational status.

Proposed Solution

To address the requirements outlined in the problem definition, we propose a distributed system architecture leveraging modern tools and technologies to handle structured and unstructured data efficiently. The architecture of the proposed solution is illustrated in Figure .

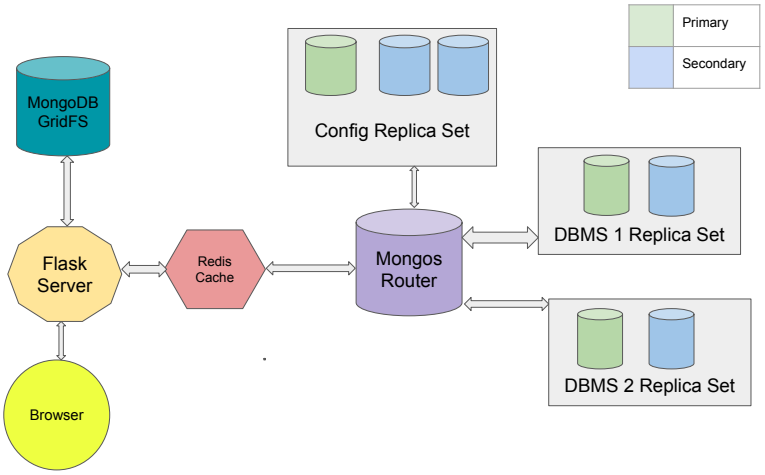


Figure 3:Architecture of the proposed solution

The architecture includes the following key components:

- A **Mongos Router** manages queries and coordinates the distributed databases to ensure high availability and scalability.
- Two **Replica Sets** (DBMS1 and DBMS2) handle the fragmented structured data based on the defined allocation scheme.
- A **Config Replica Set** stores metadata and maintains the overall system configuration.
- A **Redis Cache** acts as a high-performance layer to reduce latency for frequently accessed data.
- **MongoDB GridFS** stores and manages unstructured data such as multimedia files associated with articles.
- A **Flask Server** serves as the interface between the browser (frontend) and the backend database operations.

Tools and Technologies Used

Docker

Docker is a containerization platform that allows applications to run in lightweight, isolated environments called containers. Containers provide a consistent environment, ensuring that the software works seamlessly regardless of the underlying system. Docker is particularly useful for deploying distributed systems, as it eliminates the need for separate physical servers by simulating multiple independent environments on a single machine. In this project, Docker is used to simulate a distributed MongoDB sharded cluster, a Redis cache layer, and a Flask application. Docker Compose simplifies the orchestration of these containers, allowing services to be started and managed with a single configuration file.

MongoDB

MongoDB is a document-oriented NoSQL database that provides flexibility in handling both structured and unstructured data. Its features make it highly suitable for distributed database systems:

- **Sharding:** MongoDB supports horizontal scaling through sharding, where data is automatically distributed across multiple shards (servers). The system's query router (`mongos`) determines which shard holds the requested data, ensuring efficient data retrieval and balancing the load across servers.
- **Replica Sets:** Replica sets ensure high availability and fault tolerance by maintaining copies of the data across multiple servers. If the primary server fails, one of the secondary servers is promoted to primary, ensuring uninterrupted operations.
- **GridFS:** GridFS is a specification in MongoDB for storing and retrieving large files such as images, audio, or video. Unlike traditional file storage systems, GridFS breaks files into smaller chunks and stores them in collections, enabling efficient storage and retrieval. This project uses GridFS to manage multimedia data, making it a better alternative to Hadoop HDFS for handling large unstructured files.
- **PyMongo:** PyMongo is a Python library for interacting with MongoDB. It provides an interface to execute database operations, such as queries, updates, and configurations, programmatically.

Redis

Redis is an in-memory data structure store that serves as a caching layer to enhance performance. By storing frequently accessed data in memory, Redis reduces latency and database load, ensuring quicker responses to client requests. Key features utilized in this project include:

- **In-Memory Caching:** Redis stores a subset of the data in memory for faster retrieval. For instance, frequently queried data from MongoDB is cached in Redis to minimize database calls.
- **LRU Cache Policy:** A Least Recently Used (LRU) policy is implemented to manage memory usage efficiently. When the cache reaches its memory limit, the least recently used data is evicted to make room for new entries.
- **Cache Consistency:** Redis is integrated with MongoDB to maintain cache consistency. Whenever a document in MongoDB is updated or deleted, the cache is automatically updated to reflect these changes.

Flask

Flask is a lightweight Python web framework used for developing web applications and APIs. In this project, Flask serves as the intermediary layer between the client (browser) and the backend systems. It handles incoming HTTP requests, processes them using the MongoDB database or Redis cache, and sends back the appropriate responses. Flask's simplicity and flexibility make it ideal for integrating multiple backend components, such as the MongoDB sharded cluster and Redis cache layer.

Integration of Technologies

The integration of these tools and technologies forms a robust and scalable distributed system:

- Docker orchestrates the deployment and execution of the MongoDB sharded cluster, Redis cache, and Flask server, ensuring a seamless multi-container environment.
- MongoDB manages data storage and retrieval with advanced features like sharding for scalability, replica sets for fault tolerance, and GridFS for large file handling.
- Redis acts as a caching layer, reducing database load and ensuring quick responses for frequently accessed data.
- Flask provides an interface for clients to interact with the system, handling requests and coordinating between MongoDB and Redis.

This architecture ensures high availability, fault tolerance, and efficient data management, making it suitable for large-scale distributed applications.

Sharded MongoDB Cluster

A sharded MongoDB cluster is a distributed database architecture designed to manage large datasets and provide horizontal scaling and high availability. This section outlines the key components and architecture of the sharded MongoDB cluster implemented in this project.

Architecture Overview

The sharded MongoDB cluster consists of three primary components: Config Servers, Shards, and the Query Router (`mongos`). Each component plays a specific role in ensuring data distribution, scalability, and fault tolerance.

Config Servers

Config servers store metadata and configuration information for the cluster, including the mapping of data chunks to shards. This metadata is essential for the query router to determine the location of data across the cluster. The cluster includes a replica set of three config servers (`config1`, `config2`, `config3`) to ensure high availability and fault tolerance. If one config server becomes unavailable, the others maintain the cluster's operation without interruption.

Shards

Shards are the primary data storage units in a sharded cluster. Each shard is responsible for storing a subset of the overall dataset. The shards in this architecture are organized as replica sets to ensure data redundancy and fault tolerance. The cluster contains two shards:

- **Shard 1:** Comprises two replicas (`shard1_replica1` and `shard1_replica2`).

- **Shard 2:** Comprises two replicas (`shard2_replica1` and `shard2_replica2`).

The use of replica sets ensures that data remains available even if one of the replicas fails.

Query Router (`mongos`)

The query router is the interface between client applications and the sharded cluster. It routes client queries to the appropriate shards based on the metadata stored in the config servers. The `mongos` router simplifies client interaction by abstracting the complexity of the distributed architecture. Clients interact with the cluster as if it were a single database, and the router ensures that queries are efficiently distributed to the correct shards.

Loading Reads, Users, and Articles Tables

This section outlines the process of loading and sharding the `reads`, `users`, and `articles` collections in MongoDB. The steps include enabling sharding, configuring shard keys, and assigning shard zones.

Steps for Loading and Sharding Collections

The sharding of collections follows these main steps:

1. Enable sharding for the target database.
2. Create an index for the collection to be sharded. This index will serve as the shard key, which can be a single field or multiple fields. The shard key is critical for distributing data evenly across shards.
3. Enable sharding for the collection and set the created index as the shard key.
4. Configure the sharding zones (zone tags and zone ranges) for each shard, specific to the collection. This determines where each document in the collection will be assigned.
5. Ensure that the balancer is running for the collection to maintain proper distribution.

Loading and Sharding the `users` Collection

- The `users` collection is sharded based on the compound key {`region, uid`}. This ensures that data is distributed according to the user's `region`.
- A unique index is created on {`region, uid`}:

```
db.users.createIndex({"region": 1, "uid": 1})
```

- Sharding is enabled for the `users` collection, and the shard zones are configured as follows:
  - `Beijing` region data is assigned to `shard1ReplSet`.
  - `Hong Kong` region data is assigned to `shard2ReplSet`.
- Data is divided using zone ranges:

```
sh.addTagRange(
    "readersDb.users",
    {"region": "Beijing", "uid": MinKey},
    {"region": "Beijing", "uid": MaxKey},
    "Beijing"
)
sh.addTagRange(
    "readersDb.users",
    {"region": "Hong Kong", "uid": MinKey},
    {"region": "Hong Kong", "uid": MaxKey},
    "Hong Kong"
)
```

- The balancer is enabled for the collection to maintain proper distribution.

Loading and Sharding the `articles` Collection

- The `articles` collection is sharded based on the compound key {`category, aid`}, where `category` refers to the article type (`science` or `technology`).
- A unique index is created on {`category, aid`}:

```
db.articles.createIndex({"category": 1, "aid": 1})
```

- Sharding is enabled for the `articles` collection, and the shard zones are configured as follows:
  - `science` articles are assigned to `shard1ReplSet`.
  - `technology` articles are assigned to `shard2ReplSet`.
- Data is divided using zone ranges:

```
sh.addTagRange(
    "readersDb.articles",
    {"category": "science", "aid": MinKey},
    {"category": "science", "aid": MaxKey},
    "science"
)
sh.addTagRange(
    "readersDb.articles",
    {"category": "technology", "aid": MinKey},
    {"category": "technology", "aid": MaxKey},
    "technology"
)
```

- The balancer is enabled for the collection to maintain proper distribution.

Loading and Sharding the `reads` Collection

- The `reads` collection is sharded based on the compound key {`region, id`}. Since `reads` depends on data from `users` and `articles`, preprocessing is required:
  1. The `users` collection is projected to extract {`uid, region`}, and the results are stored in a temporary collection `uid_reg`.
  2. The `reads_unsharded` collection is updated with the `region` field by performing a `lookup` with `uid_reg`.
  3. Similarly, `articles` are projected to extract {`aid, category, timestamp`}, and the results are stored in `aid_cat_ts`.
  4. The `reads_unsharded` collection is updated with `category` and `article_timestamp` fields by performing a `lookup` with `aid_cat_ts`.
- After preprocessing, a unique index is created on {`region, id`}:

```
db.reads.createIndex({"region": 1, "id": 1})
```

- Sharding is enabled for the `reads` collection, and the shard zones are configured as follows:
  - `Beijing` region data is assigned to `shard1ReplSet`.
  - `Hong Kong` region data is assigned to `shard2ReplSet`.
- Data is divided using zone ranges:

```
sh.addTagRange(
    "readersDb.reads",
    {"region": "Beijing", "id": MinKey},
    {"region": "Beijing", "id": MaxKey},
    "Beijing"
)
sh.addTagRange(
    "readersDb.reads",
    {"region": "Hong Kong", "id": MinKey},
    {"region": "Hong Kong", "id": MaxKey},
    "Hong Kong"
)
```

- The balancer is enabled for the collection to maintain proper distribution.

While the sharding process was implemented successfully, WE encountered the following limitations:

1. **Duplicate Data for Sharding:** MongoDB does not support assigning the same data to multiple shards. To meet this requirement for `sci_articles`, the science articles were duplicated into a separate collection.
2. **Dependency on Shard Keys:** Since `reads` depends on `region` from `users` and `category` from `articles`, additional fields were added to `reads` to enable sharding. This introduces extra storage overhead.

Populating the Popular Rank and Be-Reads Tables

In this section, we explain the approach taken to populate the *Popular Rank* and *Be-Reads* tables in the distributed MongoDB setup. The focus is on utilizing aggregation pipelines for data transformation and implementing sharding configurations for efficient data distribution.

Populating Be-Reads Table

The *Be-Reads* table aggregates user interactions with articles, such as reads, comments, agreements, and shares. These interactions are sourced from the `reads_unsharded` collection and are grouped by article identifiers. The process is as follows:

1. Extract relevant fields: Filter and transform input data to extract user interactions (e.g., read, comment, agree, share).
2. Compute aggregated metrics: Use MongoDB's `$group` stage to compute counts (e.g., total reads, comments) and generate lists of user IDs corresponding to each type of interaction.
3. Format article IDs: Modify the structure of article IDs for consistency and improved readability.
4. Store results: Output the aggregated data into an intermediate collection, `bereads_unsharded`.
5. Sharding and duplication: Based on the article categories, shard the *Be-Reads* data between DBMS1 and DBMS2. Articles under the `science` category are duplicated across both DBMS1 and DBMS2, while those in the `technology` category are allocated only to DBMS2.

Populating Popular Rank Table

The *Popular Rank* table is designed to rank articles by their popularity over different temporal granularities (daily, weekly, and monthly). The steps include:

1. Extract timestamps and calculate scores:
   - Parse timestamps into date components (year, month, day, and week).
   - Compute a popularity score for each article based on user interactions (read, comment, agree, and share counts).
2. Group data by temporal granularity:
   - Aggregate data for daily, weekly, and monthly intervals using distinct pipelines.
   - Rank articles within each interval by their aggregated popularity scores.
3. Store top-ranked articles:
   - Select the top 10 articles for each interval and store them in intermediate collections.
   - Merge the results from daily, weekly, and monthly rankings into a unified `pop_ranks_unsharded` collection.
4. Sharding by temporal granularity:
   - Allocate daily rankings to DBMS1.
   - Allocate weekly and monthly rankings to DBMS2.

Sharding Considerations

MongoDB sharding does not allow overlapping data across shards due to replica set management. As a result, duplicate collections, such as `sci_bereads`, are created to handle cases where data needs to exist in multiple shards (e.g., science articles in *Be-Reads*).

Using GridFS for File Storage

GridFS is a robust file storage system integrated into MongoDB, tailored for managing files that exceed the BSON document size limit of 16 MB. This capability is essential for storing and efficiently retrieving large files, such as images, videos, and audio.

GridFS operates by splitting large files into smaller, manageable chunks and storing them as documents in a MongoDB database. The working mechanism is as follows:

- Files are divided into chunks, typically up to 255 KB in size, allowing for memory-efficient storage and retrieval.
- Metadata about files is stored in the `fs.files` collection, while the binary chunks of files are stored in the `fs.chunks` collection.
- GridFS uses indexing to ensure efficient retrieval. A compound index on `files_id` and chunk order enables reassembly of file chunks.
- The system scales horizontally, distributing file chunks across nodes to maintain performance and high availability.

Uploading Files to GridFS

To handle bulk file uploads, we implemented a Python script:

- It connects to the `readersDb` database and initializes a GridFS instance.
- The script processes files from a specified directory, checking if each file already exists in GridFS before uploading.
- If a file does not exist, it is uploaded in binary format, and its metadata is stored in `fs.files`.
- The script logs the status of each upload for easy tracking and debugging.

This approach ensures that files are uploaded efficiently, avoiding redundancy and managing metadata automatically.

Retrieving Files from GridFS

File retrieval is managed by a web API that:

- Fetches file metadata from `fs.files` based on the requested filename.
- Retrieves the file's binary data from `fs.chunks`, reassembling chunks as needed.
- Dynamically determines the content type from the file extension and streams the file to the client.
- Converts `flv` video files to `mp4` format before streaming, ensuring compatibility with modern systems.

This streamlined process makes large files accessible in a user-friendly manner while optimizing server resources.

Why Use GridFS?

GridFS is the ideal choice for this application due to its seamless integration with MongoDB and simplicity:

- It leverages MongoDB's built-in capabilities, reducing the need for additional configuration.
- GridFS's scalable and distributed nature aligns with the project's requirements for managing large datasets efficiently.
- The built-in indexing and metadata management further enhance performance, especially for high-availability applications.

Web Application Server Overview

The web application server is a Flask-based system that serves as the primary interface for interacting with the distributed database. It facilitates user management, article handling, popularity ranking, and file streaming while incorporating caching mechanisms for performance optimization. Below, we provide a high-level overview of its design and functionality.

System Architecture

The application server is connected to multiple backend components, including:

- **MongoDB Databases:** The primary and media databases store user, article, and popularity ranking data. GridFS is used for managing large media files.
- **Redis Caching Layer:** Redis is integrated as an intermediary caching layer to enhance performance. It stores frequently accessed data to minimize database queries. Updates and deletions in the database are synchronized with the cache to maintain consistency.

Key Functionalities

The server provides several core features to support application requirements:

User Management Users can be added, edited, and deleted through dedicated routes. User data, such as name, email, and preferences, is stored in the MongoDB database. User interfaces are dynamically generated using templates.

Article Management Articles are handled through routes that allow searching, adding, editing, and deleting. Article data includes metadata, text content, and associated media files. Media files are uploaded and retrieved using GridFS.

Popularity Ranking The server processes and displays popularity rankings for articles based on temporal granularities (daily, weekly, and monthly). Rankings are fetched from the database and linked to corresponding articles for display. Human-readable timestamps are generated for user-friendly views.

File Streaming Media files, such as images and videos, are streamed directly to the client. The server retrieves file metadata and chunks from GridFS and reassembles them for streaming. Video files in `flv` format are converted to `mp4` before streaming for compatibility.

Caching Mechanism

To improve response times and reduce database load, the server employs Redis for caching:

- Frequently accessed records are stored in Redis. If a record is not found in the cache, it is fetched from the database and added to Redis for future use.
- Cache synchronization is ensured during updates and deletions. For example, when a document is updated or deleted in MongoDB, the corresponding cache entry is also updated or removed.

Background Jobs for Derived Collections

We have two scripts that ensure the derived collections in the database remain up-to-date with the latest changes in target collections. These scripts operate continuously to monitor CRUD (Create, Read, Update, Delete) operations and apply necessary updates to maintain data consistency and accuracy. Below, we describe the functionality and purpose of two such background jobs.

Script 1: Maintaining the *Science Articles* Collection

The first script focuses on synchronizing the `sci_articles` collection with the main `articles` collection for records categorized as *science*. The script employs MongoDB's `watch()` method to monitor changes in the `articles` collection and triggers the following actions:

- Detects changes specifically for documents with the `science` category.
- Performs an aggregation pipeline to fetch all relevant *science* articles and merges them into the `sci_articles` collection, replacing old records.

Script 2: Updating Read and Popularity Data

The second script tracks insert operations in the `reads` collection to update the `bereads`, `sci_bereads`, and `pop_ranks` collections. Its primary responsibilities include:

- Monitoring new read records and incrementally updating counters for reads, comments, agreements, and shares in the `bereads` and `sci_bereads` collections.
- Managing lists of user IDs associated with each type of interaction to prevent duplication.
- Updating popularity rankings for articles across different temporal granularities (daily, weekly, and monthly) in the `pop_ranks` collection.

Solution Evaluation

Loading Time for Data

The loading time for each table is summarized in Table 2.

Table	**Time (in seconds)**
Users	0.3
Articles	0.3
Sci Articles	0.1
Reads	15
Be-Reads	0.5
Sci Be-Reads	0.35
Pop Rank	45

Table 2: Loading Time for Data

Query Time

The query times for the selected tables under cache hit and cache miss scenarios are provided in Table 3.

Table	**Cache Hit Time (ms)**	**Cache Miss Time (ms)**
Users	5	50
Articles	7	70
Pop Rank	15	150

Table 3: Query Time for Cache Hit and Miss

Shard Distribution Analysis

The distribution of data across shards was obtained using the MongoDB command `db.<collection_name>.getS`
This command provides information about:

- Data size and document count on each shard.
- Number of chunks allocated to each shard.
- Percentage of data and documents stored on each shard.
- Average object size for each shard.

For each collection, the results are summarized in the following table.

Collection	**DBMS1 Docs**	**DBMS1 Data (%)**	**DBMS2 Docs**	**DBMS2 Data (%)**
Users	6013	59.96%	3987	40.03%
Sci Articles	0	0%	4467	100%
Reads	572905	57.13%	427095	42.86%
Be-Reads	4467	44.61%	5533	55.38%
Sci Be-Reads	0	0%	4467	100%
Pop Ranks	117	84.73%	21	15.26%

Table 4: Shard Distribution for Collections

Monitoring of Collections using Compass

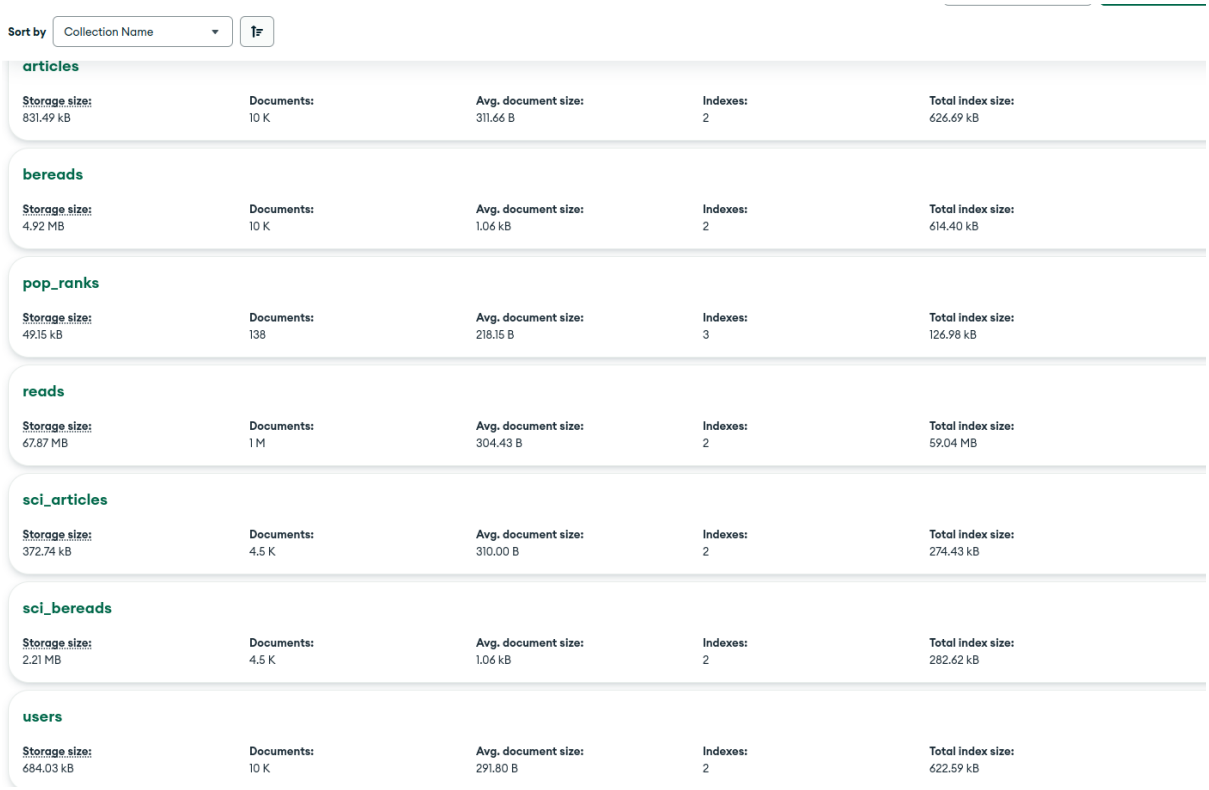
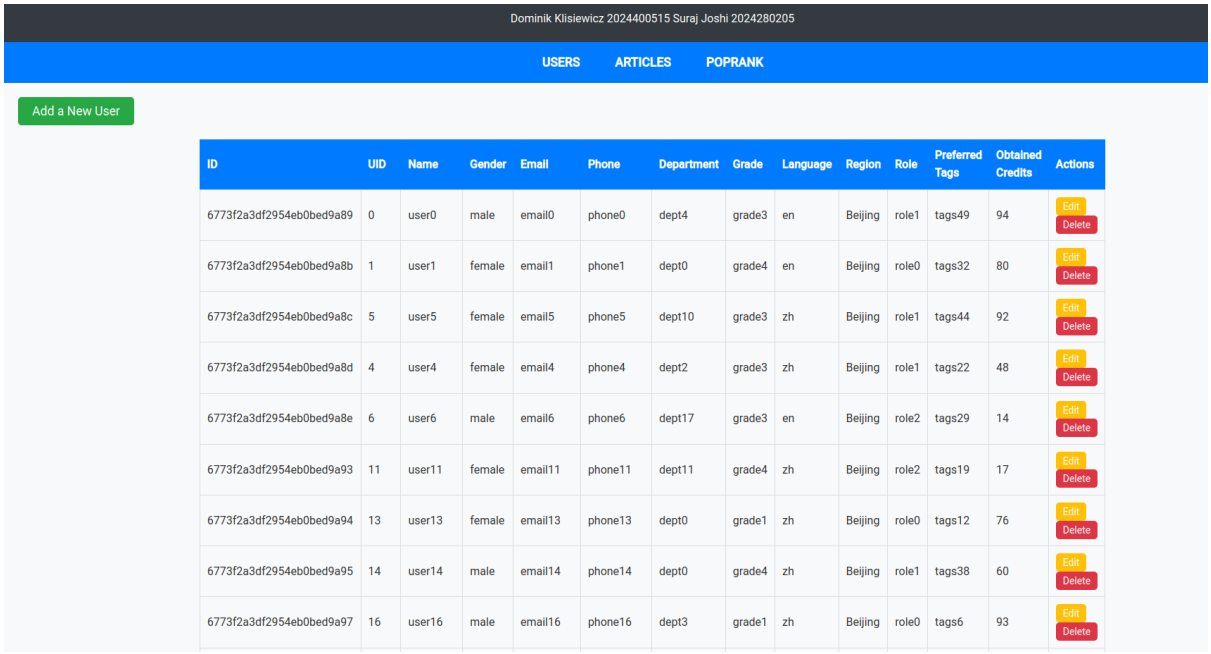


Figure : Monitoring of Collections using Compass

UI View of Users Table



ID	UID	Name	Gender	Email	Phone	Department	Grade	Language	Region	Role	Preferred Tags	Obtained Credits	Actions
6773f2a3df2954eb0bed9a89	0	user0	male	email0	phone0	dept4	grade3	en	Beijing	role1	tags49	94	Edit Delete
6773f2a3df2954eb0bed9a8b	1	user1	female	email1	phone1	dept0	grade4	en	Beijing	role0	tags32	80	Edit Delete
6773f2a3df2954eb0bed9a8c	5	user5	female	email5	phone5	dept10	grade3	zh	Beijing	role1	tags44	92	Edit Delete
6773f2a3df2954eb0bed9a8d	4	user4	female	email4	phone4	dept2	grade3	zh	Beijing	role1	tags22	48	Edit Delete
6773f2a3df2954eb0bed9a8e	6	user6	male	email6	phone6	dept17	grade3	en	Beijing	role2	tags29	14	Edit Delete
6773f2a3df2954eb0bed9a93	11	user11	female	email11	phone11	dept11	grade4	zh	Beijing	role2	tags19	17	Edit Delete
6773f2a3df2954eb0bed9a94	13	user13	female	email13	phone13	dept0	grade1	zh	Beijing	role0	tags12	76	Edit Delete
6773f2a3df2954eb0bed9a95	14	user14	male	email14	phone14	dept0	grade4	zh	Beijing	role1	tags38	60	Edit Delete
6773f2a3df2954eb0bed9a97	16	user16	male	email16	phone16	dept3	grade1	zh	Beijing	role0	tags6	93	Edit Delete

Figure : UI View of Users Table

UI View of Articles Table

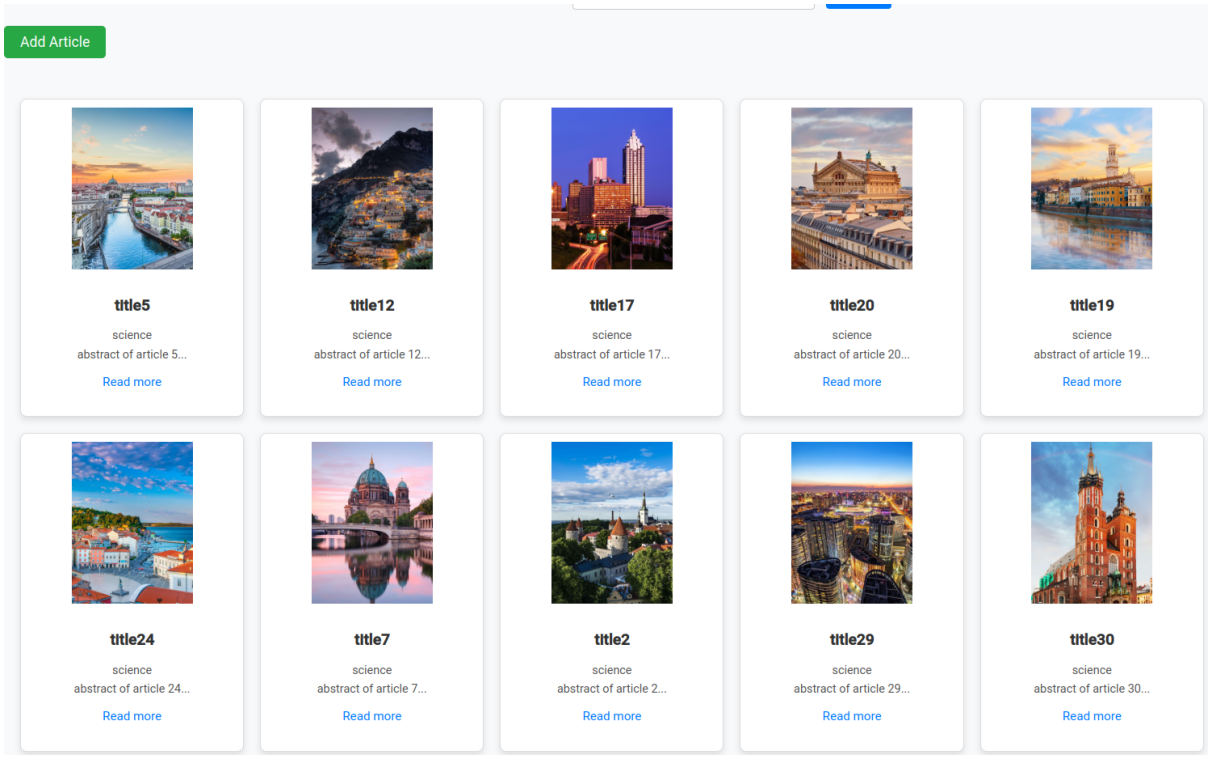


Figure : UI View of Articles Table

UI View of Popular Articles for a Given Week

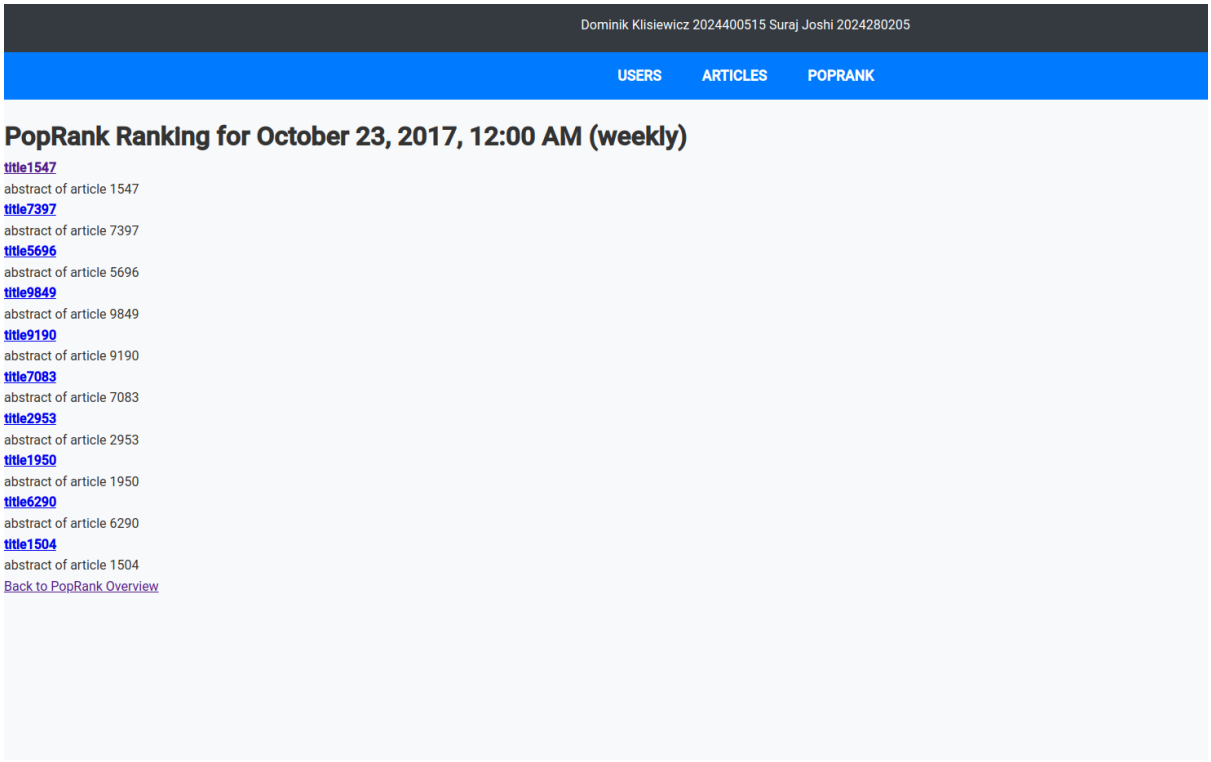


Figure : UI View of Popular Articles for a Given Week

Conclusion and Future Works

Conclusion

In this project, we implemented a sharded MongoDB cluster to manage large-scale data. The system was configured with two shard replica sets, each containing two replicas, and a configuration replica set. Data was loaded into various collections, and a thorough analysis of shard distribution was performed. The performance of data loading and query operations was evaluated, and the system's scalability and efficiency were demonstrated. Additionally, UI views and monitoring tools were utilized to manage and visualize the data, ensuring a user-friendly experience.

Future Enhancements

The following enhancements are proposed to further improve the system's robustness, scalability, and functionality:

1. **Fault Tolerance:** Implementing hot and cold standby DBMSs to ensure system reliability in case of failures.
2. **Scalability:** Allowing expansion at the DBMS level to integrate new DBMS servers seamlessly.
3. **Flexibility:** Facilitating the removal of DBMS servers without affecting the overall system.
4. **Data Migration:** Enabling efficient data migration between data centers to optimize performance and storage utilization.

5. **AI-Based Enhancements:**
   (a) *Intelligent Monitoring:* Utilizing AI-driven monitoring tools to detect anomalies, predict system failures, and optimize resource allocation.
   (b) *Smart Routing:* Leveraging AI algorithms to route queries dynamically based on workload distribution, latency, and other performance metrics.

These enhancements aim to create a more resilient, adaptive, and intelligent data management system, paving the way for future innovations in database management and analytics.

Workload Division

This project was a collaborative effort undertaken by the following team members: **Suraj Joshi** (Student ID: 2024280205) and **Klisiewicz Dominik Stanislaw** (Student ID: 2024400515). The contributions of each member are detailed below:

Suraj Joshi (2024280205)

- Designed the overall architecture for the project.
- Set up the Docker Compose environment and wrote bash scripts to automate:
  - Setting up the sharded MongoDB cluster.
  - Loading data into the clusters.
- Created background scripts for loading derived tables.
- Developed endpoints to fetch data from the Grid File System.
- Integrated a Redis caching layer to optimize query performance.
- Authored the project report and user manual.

Klisiewicz Dominik Stanislaw (2024400515)

- Set up the Grid File System (Grid FS) and loaded data into it.
- Developed the frontend and Flask application server.
- Conducted thorough testing of the application.
- Reviewed and provided feedback on the project report and user manual.

References

1. `https://www.mongodb.com/docs/manual/sharding/`
2. `https://flask.palletsprojects.com/en/stable/`
3. `https://www.mongodb.com/docs/manual/core/gridfs/`
4. `https://docs.docker.com/compose/`
5. `https://www.docker.com/`

Appendix

GitHub Repositories

1. **Sharded Database Infrastructure Code:** This repository contains the complete codebase for setting up the sharded MongoDB infrastructure, including Docker Compose scripts, data loading scripts, and configuration for sharding. `https://github.com/nonlinearjunkie/Mocking-Distributed-Databases`

2. **Web Application Code:** This repository hosts the code for the web application, including the frontend, Flask server implementation, and integration with the sharded database and caching layers. `https://github.com/DominikKlisiewicz/database_UI`